QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Content

Document Control

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability 3
2. Risk Assessment & Mitigation Strategy 4-5
3. Test Environment & Execution Strategy 6
4. CI/CD Release Thresholds & Automation Gates 7-8
5. Test Case Specifications (Drafts) 8-10
6. Test Strategy & Plan Sign-Off 10-13

© UMHackathon 2026

Email: umhackathon@um.edu.my

Document Control

Field	Detail
System Under Test (SUT)	*UMHackathon 2026 – Team Last Minute*
Team Repo URL	*https://github.com/TeyYongZhun/Last_Minute_ChatRojak*
Project Board URL	*https://sharing.clickup.com/90182651037/b/h/2kzmw34x-318/d680f4f30fc81b5*
Live Deployment URL (optional for preliminary round)	-

Objective:

The primary objective is ensure Last Minute ChatRojak achieve high reliability and automation by perfecting its proprietary AI pipeline. The core objective is to autonomously and accurately convert raw, unstructured chat history (fragmented thoughts, colloquial language) into a fully structured, categorized, and prioritized task list, eliminating manual oversight. This seamless, intelligent layer will transform casual conversation directly into an organized, executable project plan.

PRELIMINARY ROUND (Test Strategy & Planning)

1. Scope & Requirements Traceability

*This section aligns the testing connected back to specific user requirements via "**Requirement Traceability Matrix".** So, it ensures every test has been conducted to prevent bugs/failure in products for that specific requirements and unplanned feature creep.*

1.1 In-Scope Core Features

- AI-Powered Task Extraction: The system's AI reads through an entire unstructured chat log and automatically pulls out specific tasks, deadlines, and important details.
- Intelligent Prioritization: The AI automatically analyzes each task and sorts it by urgency and importance, deciding if it's something to "Do Now," "Plan" for later, a "Quick Win," or something to do "Later."
- Power Grid Dashboard: Displays all the tasks visually in a four-quadrant grid, so we can see exactly what needs to be done at a glance. It also shows which tasks depend on others.

© UMHackathon 2026

Email: umhackathon@um.edu.my

1.2 Out-of-Scope

- Real-time voice note parsing (audio-to-text integration).
- Complex multi-tenant Role-Based Access Control (RBAC).
- Automated replying directly within the native WhatsApp environment.

2. Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood (1–5)	Severity (1–5)	Risk Score (L×S)	Mitigation Strategy	Testing Approach
OpenAI / Anthropic API Rate Limits or Downtime	*3*	*5*	*15 (High)*	*Return a graceful error message urging the user to try again later*	*Force network failure to API and verify the fallback UI message.*
AI hallucinations or formatting errors crashing the database	*3*	*5*	*15 (High)*	*Utilize validator.js as middleware to catch bad JSON and request a retry.*	*Inject malformed JSON to trigger validator.js and monitor retry handling.*
Parsing succeeds but planning module fails	*2*	*4*	*8(Medium)*	*Bypass the planner and insert raw tasks into an "Unsorted" bucket.*	*Force failure in task2Planner.js and check SQLite insertion.*

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*
5	*Almost Certain*	*5*	*Critical Failure of the system*

Risk Score = Likelihood × Severity

© UMHackathon 2026

Email: umhackathon@um.edu.my

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Test Environment & Execution Strategy

- **Unit Test**
    - *Scope: task1Parser.js, task2Planner.js, and validator.js logic.*
    - *Execution: Vites framework during development and in the CI pipeline.*
    - *Isolation: Mock OpenAI/Anthropic APIs and better-sqlite3 database.*
    - *Pass Condition: Outputs perfectly match required strict JSON schemas.*
- **Integration Test**
    - *3-step AI pipeline orchestration (promptChain.js with Parser, Planner, Validator) and Frontend-to-Backend /process API.*
    - *Execution: Performed after unit testing, prior to merging features into the main branch.*
    - *Workflow: Ensure raw text successfully passes through all 4 AI modules without mocking.*
    - *Pass Condition: Pipeline returns a final JSON object matching the exact expected schema without errors.*
- **Test Environment (CI/CD Practice):**
    - *Local Testing: Manual execution on developer localhost (http://localhost:8000).*
    - *Staging/CI: GitHub Actions triggers Jest/Vitest test suite on every PR pushed to main.*
    - *CI/CD Automated Pipeline: Workflow runs npm install and npm test, blocking PR merges if any test fails.*
- **Regression Testing & Pass/Fail Rules:**
    - *Execution Phase: Automated suite runs across the application every time a PR targets the main branch.*
    - *Pass/Fail Condition: Output JSON schema must perfectly align with expectations. Any structural mismatch is a fail.*
    - *Continuation Rule: CI/CD pipeline enforces a strict block; failed regression tests prevent merging to main.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Test Data Strategy:**
    - *Manual: 5-10 static fixture files containing varied chat scenarios (simple, complex deadlines, long conversations, casual chat).*
    - *Automated: Scripts feed fixture files into the API endpoint and compare outputs against stored JSON snapshots.*
- **Passing Rate Threshold:**
    - *Standard Rate: Minimum 95% pass rate across all general test cases.*
    - *For the critical tests, 100% pass rate required for core Integration and Regression tests. No exceptions allowed.*

4. CI/CD Release Thresholds & Automation Gates

4.1 Integration Thresholds (Merging to Main)

The primary needs of this threshold to be crucial is because the main branch must be a stable source of truth. So, it needs to be achieved through continuous integration checks.

Checks	***Requirements***	***Project***	***Pass/Failed***
Automatic Build	***Zero Build Error***	***5 Errors***	***Failed***
Unit Tests	***100% Passing Rate***	***100%***	***Pass***
Code Quality	***Zero Linting Error***	***97%***	***Failed***
Test Coverage	***Minimum 81.2%***	***79%***	***Failed***

© UMHackathon 2026

Email: umhackathon@um.edu.my

4.2 Deployment Thresholds (Pushing to Production)

Checks	***Requirements***	***Project***	***Pass/Failed***
Regression Test	***Minimum 90%***	***92.8%***	***Pass***
AI Output Pass Rate	***Minimum of 80% of documented prompt***	***80%***	***Pass***
Critical Bugs	***Zero P0/01 Bugs***	***2***	***Failed***
API Performance	***Response Time < 800ms***	***>5000ms***	***Failed***
Security	***API keys are not exposed***	***Clean***	***Pass***

5. Test Case Specifications

The test cases MUST include at least:

- ***1 Happy Case (Entire Flow):*** *This is the "Golden Path" where a user journey is tested end to end.*
- ***1 Specific Case (Negative/Edge Test):*** *This case should portray a scenario which triggers error in the system and the system handles it with proper handling techniques.*
- **2 Non-Functional (NFR) Tests (Performance/Load)*:*** *This test consists of non-feature-based tests but architectural tests.*

Test Case ID	**Test Type & Mapped Feature**	**Test Description**	**Test Steps**	**Expected Result**	**Actual Result**
TC-01	***Happy Case (Entire Flow):*** *AI Task Processing*	*Verify a user's chat message is correctly processed end-to-end by the 3-step AI chain (Extract, Plan, Validate) and saved as a structured task.*	*1. User uploads a WhatsApp .txt chat export file to the /upload API endpoint.* *2. The system parses the file and initiates the promptChain for*	*The API should return a 200 OK status with a valid JSON object containing the structured task, which is then saved to the database.*	*The end-to-end flow completes successfully. The AI correctly structures the task, and the retry/validation logic ensures a high success*

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
			each message. *3. The AI extracts, categorizes, plans, and validates tasks from the messages.*		*rate.* ***Status: Passed***
TC-02	***Specific Case (Negative):*** *AI Returns Invalid Data*	*Verify the system gracefully handles a scenario where an AI step returns non-JSON text, without crashing the server.*	*1. User uploads a chat file to the /upload API.* *2. An AI step is simulated to return plain text (e.g., "I cannot help") instead of JSON for one of the messages.* *3. The system's error handling and retry logic are triggered.*	*The server should not crash. The promptChain should retry the failed AI call. If it continues to fail, that specific message should be skipped, and the system should continue processing the rest of the file.*	*The request for the single failed message fails, but the server remains stable. The withRetry function and validator module prevent a crash, and the system successfully processes the other valid messages in the file.* ***Status: Passed***
TC-03	***NFR (Performance) :*** *API Response Time*	*Verify the end-to-end response time of the /upload API endpoint is under the required 800ms for production readiness.*	*1. Use an API testing tool to upload a standard-sized chat file to the /upload endpoint.* *2. Measure the time from request to final response.* *3. Repeat multiple times to find the average.*	*The average response time should be less than 800ms.*	*The average response time is significantly over 800ms (estimated >5000ms per message) due to the four sequential, blocking network calls to the AI model for each message in the file.* ***Status: Failed***

© UMHackathon 2026

Email: umhackathon@um.edu.my

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result
TC-04	***NFR (Security): API Key Exposure***	*Verify that the application does not expose the ILMU_API_KEY in the source code or in version control.*	*1. Scan the codebase for any hardcoded API keys.* *2. Review the gitignore file.* *3.Analyze src/client.js to confirm how the key is loaded.*	*No hardcoded keys should be found. The .gitignore file must list .env. The application must load the key from process.env*	*The API key is not hardcoded. The .gitignore file correctly excludes the .env file, and src/client.js properly loads the key from the environment variables.* ***Status: Passed***

6. AI Output & Boundary Testing

This section is designed to validate that your GLM integration does produce acceptable output and handles any inputs that are abnormal gracefully.

6.1. Prompt/Response Test Pairs

Document at least 3 Prompts/response test pairs. For each of them, do define the acceptance criteria - what a correct, acceptable or a failing response may look like for your use case.

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01	*(Extractor) A single line from a chat file:* *"Don't forget to submit the report by Friday at 2pm"*	*The output must be a valid JSON object. The isActionable field must be true. The title field must capture the core task ("submit the report"). The due field must be a valid date/time string corresponding to "Friday at 2pm".*	*json { "isActionable": true, "title": "Submit the report", "due": "2026-05-01T14:00 :00" }*	*Pass*

© UMHackathon 2026

Email: umhackathon@um.edu.my

AI-02	*(Extractor - Negative Case) A single line from a chat file:* *"lol that's so funny, see you later"*	*The output must be a valid JSON object. The isActionable field must be false. The system must not fabricate or "hallucinate" a task from a conversational message.*	*json { "isActionable": false }*	*Pass*
AI-03	*(Extractor - Task with No Time)* *A single line from a chat file:* *"remember to buy more coffee filters"*	*The output must be a valid JSON object. The isActionable field must be true. The title must be "buy more coffee filters". The due field must be null or empty, as no time was specified.*	*json { "isActionable": true, "title": "Buy more coffee filters", "due": null }*	*Pass*

6.2. Oversized/Larger Input Test

Fields	Details
Maximum Input Size	*A 500KB .txt file*
Input used while testing	*A 1MB .txt file (2x the configured limit)*
Expected Behavior	*Reject the request with a "413 Payload Too Large" error. The server should not crash and should handle the rejection gracefully.*
Actual Behavior	*The Express server correctly identifies the oversized payload before it reaches the application logic. It terminates the request and returns a 413 error to the client. The server remains stable and does not crash.*
Status	*Pass*

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.3. Adversarial/Edge Prompt Test

Test ID	Prompt Input	Expected Behaviour	Actual Behaviour(How GLM response and handle)	Status
AM-01	*(Extractor) A single line from a chat file:* *"remindr to buy mlik tomoroww also call john about the thing,,,,,,,"*	*The system must not crash. It should recognize the input is malformed and ambiguous, fail to extract a valid task, and safely ignore the message without creating a "hallucinated" or broken task entry.*	*1. The Extractor AI partially succeeds. It ignores the typos and identifies a task: "call john about the thing". However, it recognizes that "the thing" is too vague and flags the task with low confidence.* *2. The Validator step sees this low confidence flag.* *3. It triggers the Clarification Loop.* *4. A task is created with the title "Call John" and its status is set to needs_clarification.* *5. A specific question, "What 'thing' should I remind you to talk to John about?", is generated and attached to the task, prompting the user for the missing detail in the UI.*	*Pass*

© UMHackathon 2026

Email: umhackathon@um.edu.my

6.4. Hallucinating Handling

Our system treats AI output with inherent skepticism and uses a three-layered approach to ensure accuracy and prevent fabricated tasks from entering the system.

Layer 1: Strict Structural Enforcement (The Gatekeeper)

- *How it works: The very first line of defense is the extractJson function in the src/client.js file. Our system does not ask the AI for plain text; it demands a response in a strict JSON format.*
- *Mitigation: If the AI hallucinates a conversational answer (e.g., "I'm not sure what you mean, can you clarify?") instead of providing the requested JSON, the extractJson function will fail to parse it. Because of its built-in error handling, it doesn't crash. Instead, it returns an empty object. This immediately stops the hallucinated conversational response from being processed as a real task.*

Layer 2: The Dedicated Validation Step (The Fact-Checker)

- *How it works: This is the most critical layer. Our 3-step AI chain is not just "Extract, Plan" The final and most important step is Validate. Our system features a dedicated module, src/modules/validator.js, whose sole purpose is to fact-check the output from the previous AI steps.*
- *Mitigation: After the AI has generated a potential task, the validator module takes that structured data and compares it against the original user's text. It specifically checks for common hallucinations:*
- *Invented Tasks: It verifies the isActionable flag. If the Extractor AI hallucinated a task from a simple conversational message like "How are you?", the validator will check the original text, see there was no command, and override the result, flagging it as not actionable.*
- *Invented Details: It checks if the details in the task (like the title, date, or time) were actually present in the user's original message. If the user said "remind me to call mom" and the AI hallucinated a due date of "tomorrow at 5:00 PM," the validator will detect that this information was fabricated and can either remove it or flag the task for user review.*

Layer 3: The User Interface (The Final Review)

- *How it works: The final defense is the user themselves. The processed tasks are presented clearly in the Power Grid UI.*
- *Mitigation: If a subtle hallucination makes it past the automated checks, the user can immediately spot it. For example, if a task is titled "Call John about the secret project" when the user only said "Call John about the project", the user can easily edit or delete the task. This human-in-the-loop review ensures that even the most convincing hallucinations are ultimately caught.*

Important Note:

Using Agile principle, these testing will take place iteratively over the period of development, not limited to at the end.

© UMHackathon 2026

Email: umhackathon@um.edu.my